발표 주제 선정

GPU 기반 파티클 시스템 구현 및 최적화

(부제: 환경 상호작용으로의 확장)

프로젝트 개요

1) GPU 기반 파티클과 최적화



1) 파티클의 메모리 관리
2) 파티클 업데이트
3) 거리기반 LOD
4) 렌더링 기법
5) 그외

2) 환경과 상호작용하는 파티클

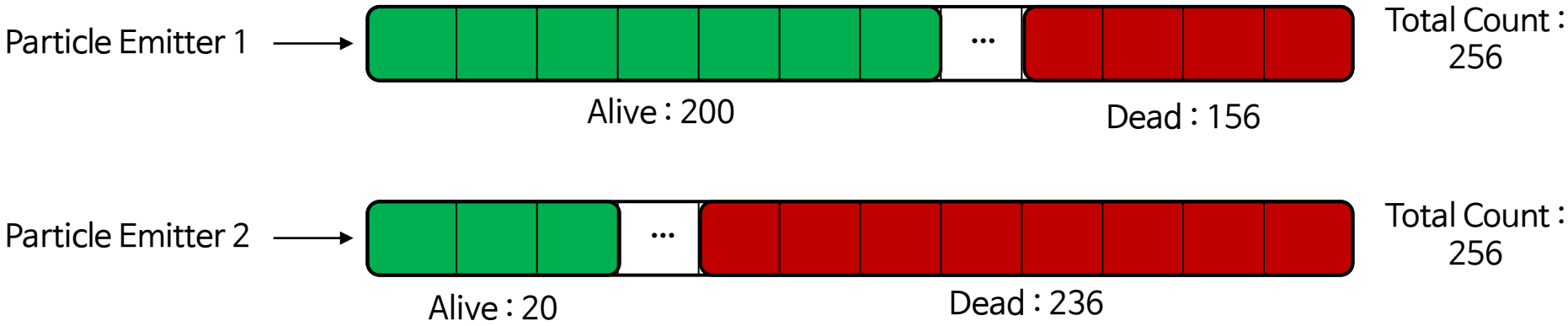


1) 지형(객체)와 충돌하는 파티클
2) 충돌한 파티클에 따라서 반응을 하는 지형(객체)

파티클의 메모리 관리

- GPU 기반 파티클과 최적화

기존

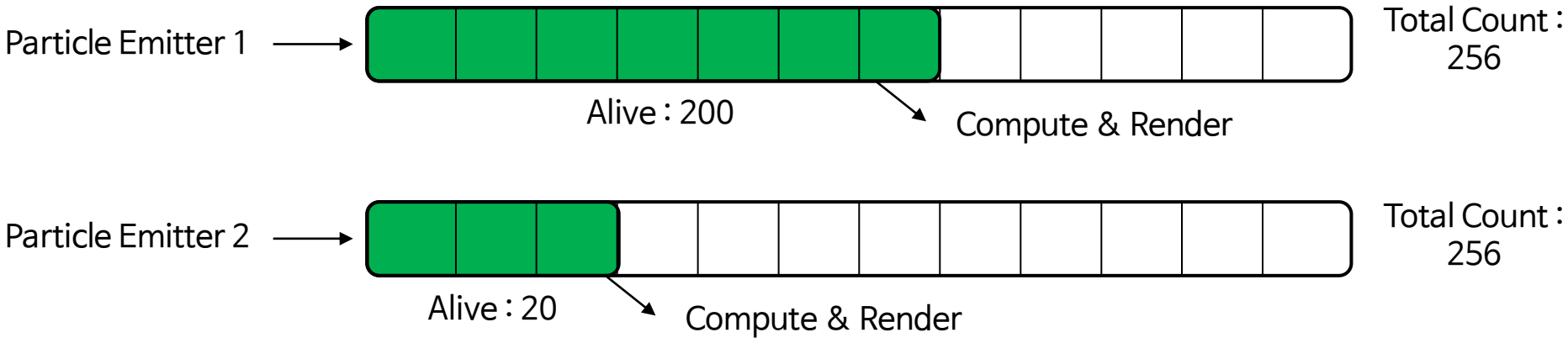


화면에 나타나지 않은 파티클(Dead) 또한 업데이트 및 렌더링의 흐름을 탄다.

파티클의 메모리 관리

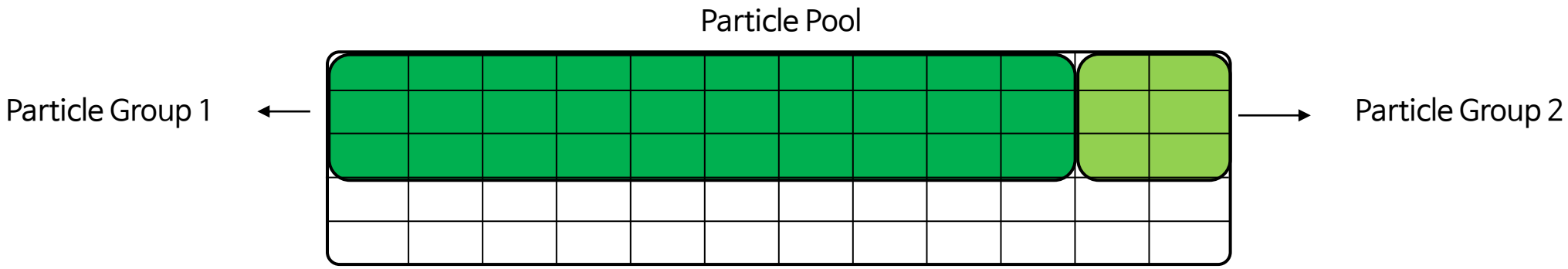
- GPU 기반 파티클과 최적화

변경1



활성화된 파티클만 업데이트 및 렌더링을 거친다.

변경2



Global Particle Pool을 통해 GPU 메모리 공간을 줄인다.

파티클 업데이트

- GPU 기반 파티클과 최적화

파티클 특성 변경

Particle

- 위치/속도
- 크기/회전
- 색상
- 수명 (LifeTime)
- ...

수명 관리 & Spawn

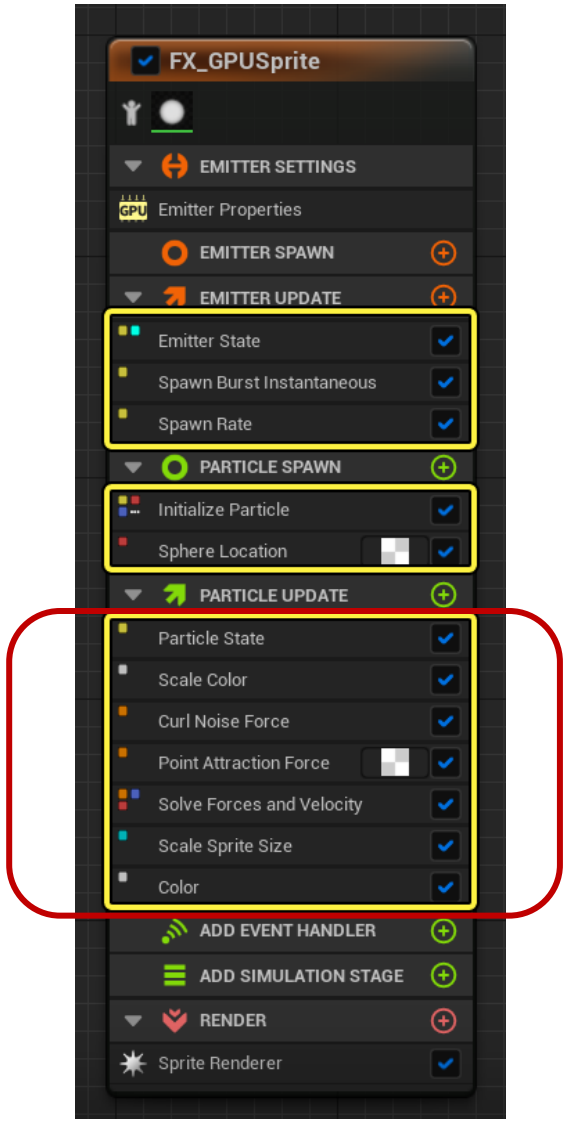
1) LifeTime이 끝났을 경우 erase 방식
2) 파티클 Spawn시 특성 초기화 & 할당 방식

그 외

1) Burst 방식
2) 파티클 애니메이션
...

파티클 업데이트

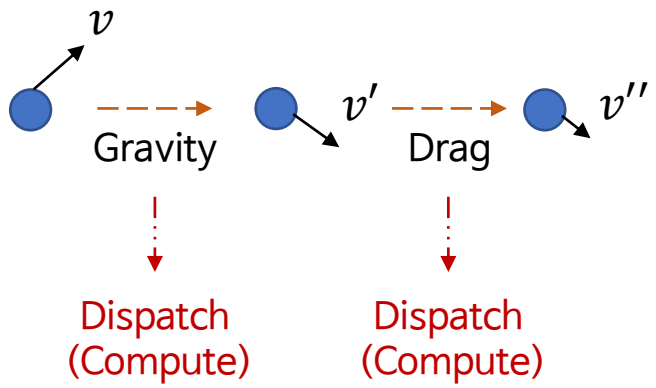
- GPU 기반 파티클과 최적화



기존

모듈 업데이트와 dispatch(compute)가 1대1 대응관계

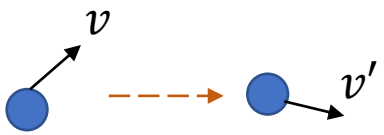
Particle Emitter
- Gravity
- Drag



변경

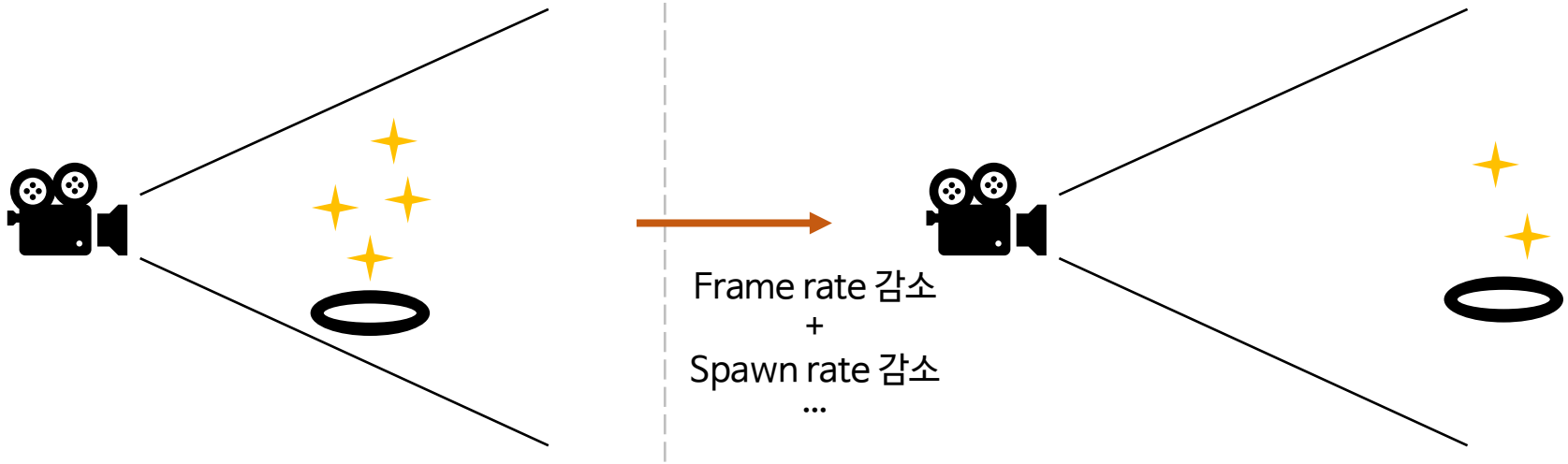
Particle Emitter
- Gravity
- Drag
- Vortex Force

한번에 dispatch

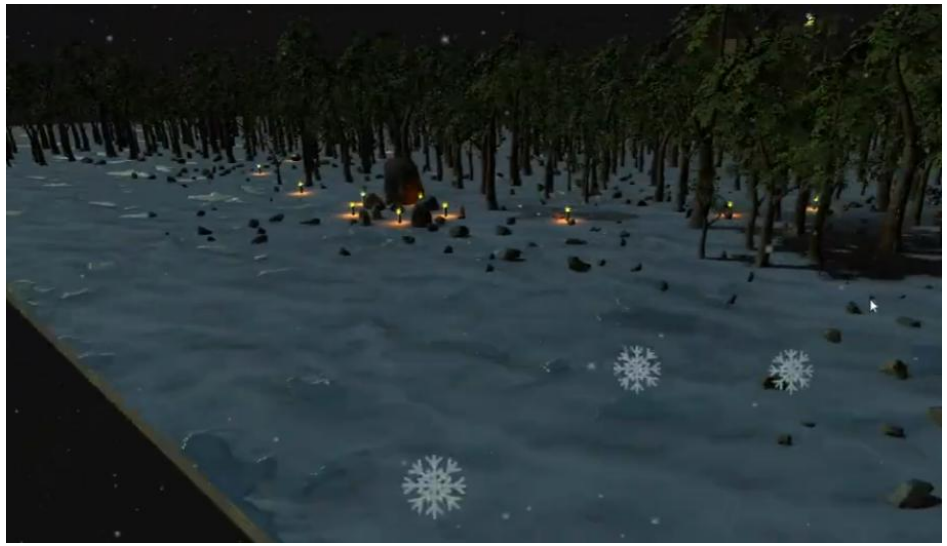


거리기반 LOD + 컬링

- GPU 기반 파티클과 최적화



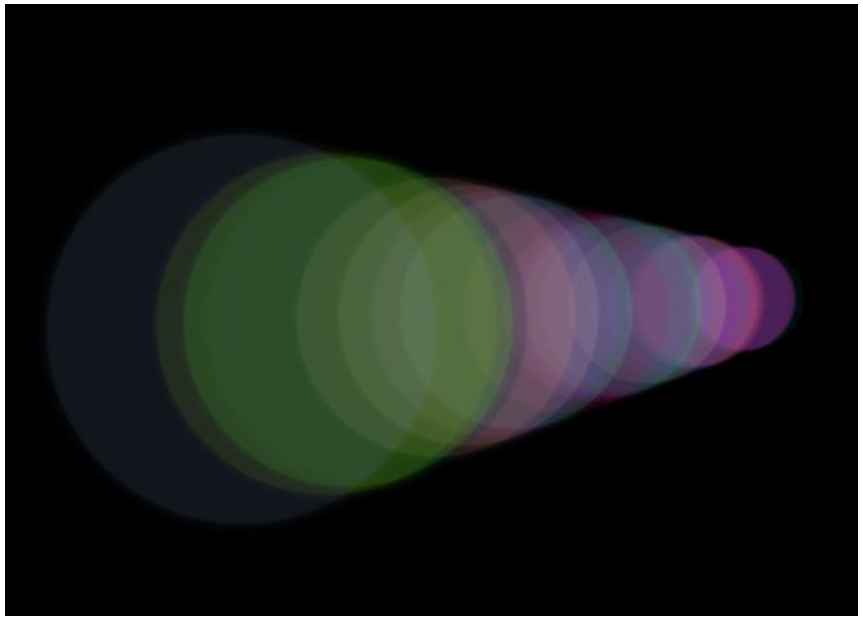


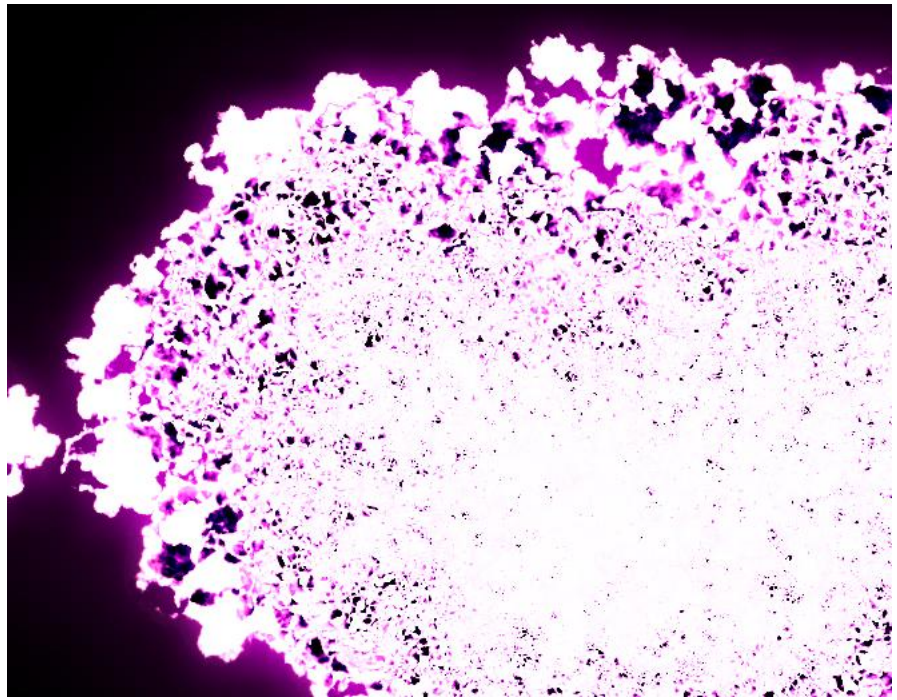


렌더링 기법

- GPU 기반 파티클과 최적화

WBOIT (**Weighted** Blended Order Independent Transparency)

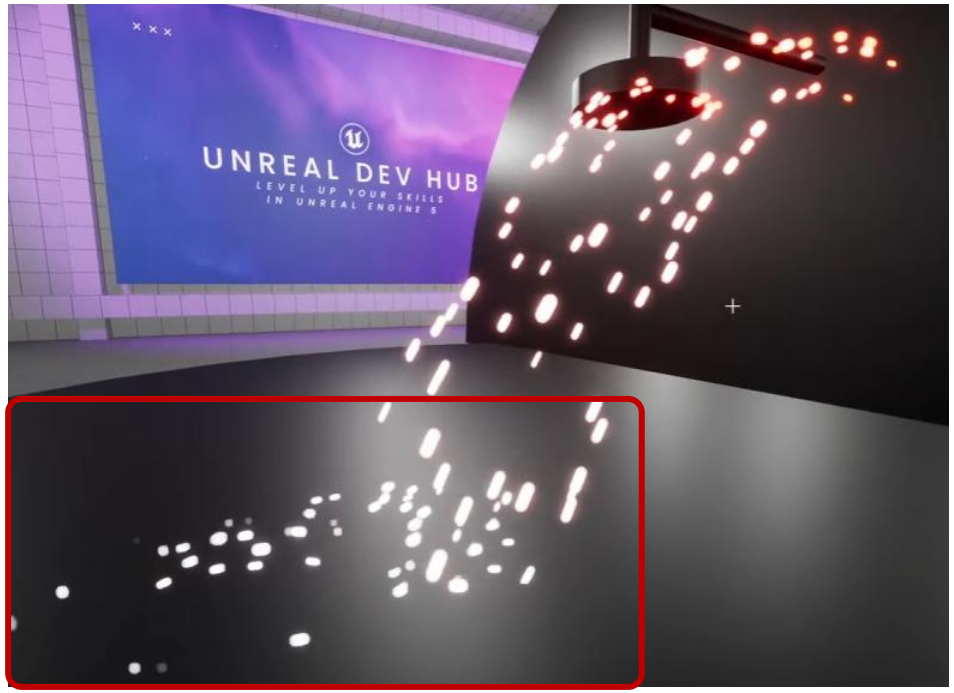




DXGI_FORMAT 값 범위 초과로 인해 일부가 검정색으로 보이게 된다.

WBOIT의 발전 방식 탐구 & 다른 렌더링 기법을 적용하여 성능 비교/렌더링 결과 비교

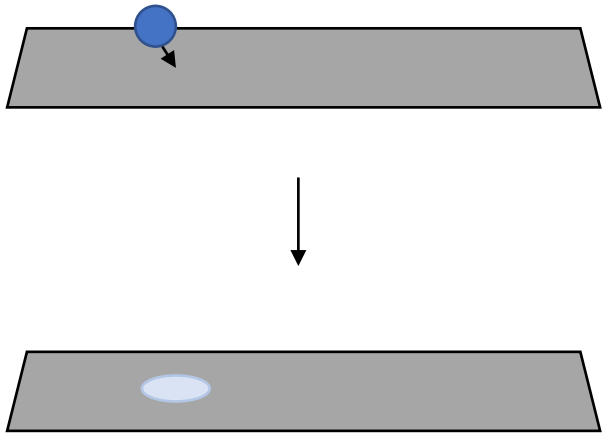
지형(객체) 충돌 파티클 - 환경과 상호작용하는 파티클



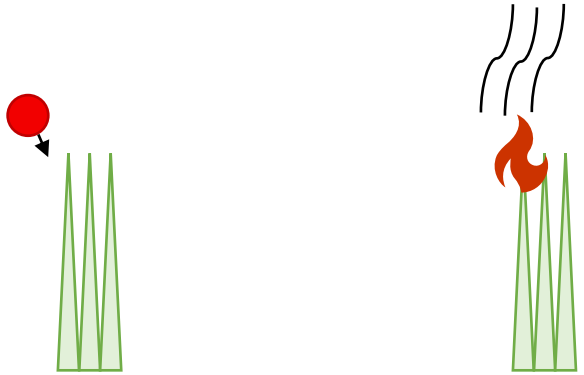
파티클과 지면(객체)이 충돌하여 파티클이 튕겨난다.
- GPU에서 지면의 정보를 알아야 한다.
- 화면(혹은 플레이어의 시야)밖에서 파티클의 충돌도 고려해야 한다.

파티클에 반응하는 지형(객체)

- 환경과 상호작용하는 파티클



데칼 생성



연쇄 반응을 일으키는 파티클

그 외



Static Mesh Location
particle



curl noise particle

구현 계획

	1주차	2주차	3주차	4주차	5주차	6주차	7주차	8주차
MiniEngine 및 dx12 학습								
파티클 메모리 설정과 관리								
파티클 업데이트 - 수명/Spawn								
파티클 업데이트 + dispatch 최적화								
거리기반 LOD + 컬링								
렌더링 기법 연구 + 다른 렌더링 기법 도입								
파티클과 지형(객체) 충돌								
파티클에 반응하는 지형(객체)								
그 외 구현 + 버퍼								